

Fast Sector Decomposition from Black-Box Symanzik Polynomials

1 The example triangle

We consider the scalar one-loop triangle with one incoming off-shell momentum and two outgoing massless momenta,

$$p_0 = p_1 + p_2, \quad p_1^2 = p_2^2 = 0, \quad p_0^2 = s = m_0^2. \quad (1)$$

All three internal propagators are taken to have the same mass m . A convenient routing is

$$D_0 = \ell^2 - m^2 + i0, \quad (2)$$

$$D_1 = (\ell + p_1)^2 - m^2 + i0, \quad (3)$$

$$D_2 = (\ell - p_2)^2 - m^2 + i0. \quad (4)$$

The scalar integral is

$$C_0(s; m^2) = \int \frac{d^D \ell}{i\pi^{D/2}} \frac{1}{D_0 D_1 D_2}. \quad (5)$$

With Feynman parameters $x_i \geq 0$, the projective representation is

$$C_0(s; m^2) = -\Gamma(\alpha) \int_0^\infty dx_0 dx_1 dx_2 \delta(1 - x_0 - x_1 - x_2) \frac{\mathcal{U}^{3-D}}{\mathcal{F}^\alpha}, \quad \alpha = 3 - \frac{D}{2}. \quad (6)$$

For this routing,

$$\mathcal{U} = x_0 + x_1 + x_2, \quad (7)$$

and

$$\mathcal{F} = m^2 \mathcal{U}^2 - s x_1 x_2 - i0. \quad (8)$$

Solving the simplex constraint by

$$x_0 = 1 - x_1 - x_2 \quad (9)$$

gives

$$0 \leq x_1 \leq 1, \quad 0 \leq x_2 \leq 1 - x_1, \quad (10)$$

and therefore

$$\mathcal{U} = 1, \quad \mathcal{F} = m^2 - s x_1 x_2 - i0. \quad (11)$$

For $D = 4 - 2\epsilon$, one has $\alpha = 1 + \epsilon$, and

$$C_0(s; m^2) = -\Gamma(1 + \epsilon) \int_0^1 dx_1 \int_0^{1-x_1} dx_2 (m^2 - s x_1 x_2 - i0)^{-1-\epsilon}. \quad (12)$$

Throughout the rest of the note, sector integrands I_s do not include the global prefactor $-\Gamma(\alpha)$. Thus an exact decomposition has the form

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_s \int_{\mathcal{C}_s} dy_s I_s(y_s). \quad (13)$$

1.1 Endpoint sectors

The primary endpoint decomposition splits the simplex into two sectors.

Sector S_1 , $x_1 \geq x_2$.

$$X_{S_1}(t, z) = (x_0, x_1, x_2) = \left(1 - t, \frac{t}{1+z}, \frac{tz}{1+z}\right), \quad 0 \leq t, z \leq 1. \quad (14)$$

The Jacobian is

$$J_{S_1}(t, z) = \frac{t}{(1+z)^2}. \quad (15)$$

The pulled-back second Symanzik polynomial is

$$\mathcal{F}_{S_1}(t, z) = m^2 - s \frac{t^2 z}{(1+z)^2} - i0. \quad (16)$$

In the massless endpoint case $m = 0$,

$$\mathcal{F}_{S_1}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2}. \quad (17)$$

The expected monomial is therefore

$$M_{S_1}(t, z) = t^2 z. \quad (18)$$

Sector S_2 , $x_2 \geq x_1$.

$$X_{S_2}(t, z) = (x_0, x_1, x_2) = \left(1 - t, \frac{tz}{1+z}, \frac{t}{1+z}\right), \quad 0 \leq t, z \leq 1. \quad (19)$$

The Jacobian is again

$$J_{S_2}(t, z) = \frac{t}{(1+z)^2}. \quad (20)$$

The pulled-back polynomial is

$$\mathcal{F}_{S_2}(t, z) = m^2 - s \frac{t^2 z}{(1+z)^2} - i0. \quad (21)$$

In the massless endpoint case,

$$\mathcal{F}_{S_2}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2}, \quad M_{S_2}(t, z) = t^2 z. \quad (22)$$

With these definitions, the two-sector representation of the original triangle is

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz I_{S_a}(t, z), \quad (23)$$

$$I_{S_a}(t, z) = \frac{t}{(1+z)^2} \mathcal{U}(X_{S_a}(t, z))^{3-D} \mathcal{F}(X_{S_a}(t, z))^{-\alpha}. \quad (24)$$

Equivalently, after inserting the explicit polynomial,

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} \left(m^2 - s \frac{t^2 z}{(1+z)^2} - i0 \right)^{-\alpha}. \quad (25)$$

For $D = 4 - 2\epsilon$ and $m = 0$, this becomes

$$C_0(s; 0) = -\Gamma(1 + \epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} G_{S_a}(t, z), \quad (26)$$

where

$$G_{S_a}(t, z) = (-s - i0)^{-1-\epsilon} (1+z)^{2\epsilon}. \quad (27)$$

1.2 Massive threshold sectors

For $m \neq 0$, the endpoint singularity is lifted. A threshold singularity appears at

$$s = 4m^2, \quad (28)$$

at the point

$$x_0 = 0, \quad x_1 = x_2 = \frac{1}{2}. \quad (29)$$

Write

$$x_0 = a, \quad x_1 = \frac{1-a+d}{2}, \quad x_2 = \frac{1-a-d}{2}, \quad (30)$$

and split

$$d = \sigma(1-a)b, \quad \sigma = \pm 1, \quad 0 \leq b \leq 1. \quad (31)$$

At threshold,

$$\mathcal{F} = m^2 [a(2-a) + (1-a)^2 b^2] - i0. \quad (32)$$

The overlap $a \sim b^2$ is blown up by two sectors for each sign σ .

Sector $T_{\sigma,1}$, $b^2 \leq a$. Use $a = r^2$, $b = ry$. Then

$$X_{T_{\sigma,1}}(r, y) = \left(r^2, \frac{(1-r^2)(1+\sigma ry)}{2}, \frac{(1-r^2)(1-\sigma ry)}{2} \right), \quad 0 \leq r, y \leq 1. \quad (33)$$

The Jacobian is

$$J_{T_{\sigma,1}}(r, y) = r^2(1-r^2). \quad (34)$$

The polynomial factorizes as

$$\mathcal{F}_{T_{\sigma,1}}(r, y) = m^2 r^2 [(2-r^2) + (1-r^2)^2 y^2] - i0. \quad (35)$$

The expected monomial is

$$M_{T_{\sigma,1}}(r, y) = r^2. \quad (36)$$

Sector $T_{\sigma,2}$, $a \leq b^2$. Use $a = b^2 y$. Then

$$X_{T_{\sigma,2}}(b, y) = \left(b^2 y, \frac{(1-b^2 y)(1+\sigma b)}{2}, \frac{(1-b^2 y)(1-\sigma b)}{2} \right), \quad 0 \leq b, y \leq 1. \quad (37)$$

The Jacobian is

$$J_{T_{\sigma,2}}(b, y) = \frac{1}{2} b^2 (1-b^2 y). \quad (38)$$

The polynomial factorizes as

$$\mathcal{F}_{T_{\sigma,2}}(b, y) = m^2 b^2 [y(2-b^2 y) + (1-b^2 y)^2] - i0. \quad (39)$$

The expected monomial is

$$M_{T_{\sigma,2}}(b, y) = b^2. \quad (40)$$

At exact threshold, the four threshold sectors reconstruct the original massive triangle as

$$C_0(4m^2; m^2) = -\Gamma(\alpha) \sum_{\sigma=\pm 1} \left[\int_0^1 dr \int_0^1 dy I_{T_{\sigma,1}}(r, y) + \int_0^1 db \int_0^1 dy I_{T_{\sigma,2}}(b, y) \right], \quad (41)$$

$$I_{T_{\sigma,1}}(r, y) = r^2(1-r^2) \mathcal{F}(X_{T_{\sigma,1}}(r, y))^{-\alpha}, \quad (42)$$

$$I_{T_{\sigma,2}}(b, y) = \frac{1}{2} b^2 (1-b^2 y) \mathcal{F}(X_{T_{\sigma,2}}(b, y))^{-\alpha}. \quad (43)$$

Here $\mathcal{U} = 1$ after the simplex constraint, so no separate power of $\mathcal{U}$ appears in the last two lines.

2 Direct approach

In the direct approach, each sector map X_s is substituted explicitly into $\mathcal{U}$ and $\mathcal{F}$. If the pulled-back polynomial factorizes as

$$\mathcal{F}_s(y) = \mathcal{F}(X_s(y)) = M_s(y) \phi_s(y), \quad (44)$$

then the sector integrand is built as

$$I_s^{\text{dir}}(y) = J_s(y) \mathcal{U}(X_s(y))^{3-D} M_s(y)^{-\alpha} \phi_s(y)^{-\alpha}. \quad (45)$$

The corresponding contribution to the original scalar integral is always understood as

$$C_0 = -\Gamma(\alpha) \sum_s \int_{\mathcal{C}_s} dy_s I_s^{\text{dir}}(y_s), \quad (46)$$

with the sector domains $\mathcal{C}_s$ specified explicitly below.

2.1 Endpoint sector S_1

For $m = 0$,

$$\mathcal{U}_{S_1}(t, z) = 1, \quad (47)$$

and

$$\mathcal{F}_{S_1}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2} = M_{S_1}(t, z) \phi_{S_1}(t, z), \quad (48)$$

with

$$M_{S_1}(t, z) = t^2 z, \quad \phi_{S_1}(t, z) = \frac{-s - i0}{(1+z)^2}. \quad (49)$$

Therefore

$$I_{S_1}^{\text{dir}}(t, z) = \frac{t}{(1+z)^2} (t^2 z)^{-\alpha} \left(\frac{-s - i0}{(1+z)^2} \right)^{-\alpha}. \quad (50)$$

For $D = 4 - 2\epsilon$, i.e. $\alpha = 1 + \epsilon$,

$$I_{S_1}^{\text{dir}}(t, z) = (-s - i0)^{-1-\epsilon} t^{-1-2\epsilon} z^{-1-\epsilon} (1+z)^{2\epsilon}. \quad (51)$$

2.2 Endpoint sector S_2

By symmetry,

$$\mathcal{F}_{S_2}(t, z) = (-s - i0) \frac{t^2 z}{(1+z)^2}, \quad M_{S_2}(t, z) = t^2 z, \quad (52)$$

and hence

$$I_{S_2}^{\text{dir}}(t, z) = (-s - i0)^{-1-\epsilon} t^{-1-2\epsilon} z^{-1-\epsilon} (1+z)^{2\epsilon}. \quad (53)$$

The full massless triangle is therefore

$$\begin{aligned} C_0(s; 0) &= -\Gamma(1+\epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz I_{S_a}^{\text{dir}}(t, z) \\ &= -2\Gamma(1+\epsilon) (-s - i0)^{-1-\epsilon} \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} (1+z)^{2\epsilon}. \end{aligned} \quad (54)$$

2.3 Threshold sectors

At $s = 4m^2$, the direct threshold integrands are

$$I_{T_{\sigma,1}}^{\text{dir}}(r, y) = r^2(1 - r^2)(r^2)^{-\alpha} (m^2 [(2 - r^2) + (1 - r^2)^2 y^2] - i0)^{-\alpha}, \quad (55)$$

$$I_{T_{\sigma,2}}^{\text{dir}}(b, y) = \frac{1}{2} b^2(1 - b^2 y)(b^2)^{-\alpha} (m^2 [y(2 - b^2 y) + (1 - b^2 y)^2] - i0)^{-\alpha}. \quad (56)$$

The dependence on σ is only in the map X_T , not in the factorized polynomial for this symmetric equal-mass triangle. These four direct threshold sectors reconstruct the threshold integral as

$$C_0(4m^2; m^2) = -\Gamma(\alpha) \sum_{\sigma=\pm 1} \left[\int_0^1 dr \int_0^1 dy I_{T_{\sigma,1}}^{\text{dir}}(r, y) + \int_0^1 db \int_0^1 dy I_{T_{\sigma,2}}^{\text{dir}}(b, y) \right]. \quad (57)$$

3 Indirect approach

The indirect approach keeps the Symanzik polynomials closed. We assume only black-box evaluators

$$\widehat{\mathcal{U}}(x_0, x_1, x_2), \quad \widehat{\mathcal{F}}(x_0, x_1, x_2), \quad (58)$$

together with their partial derivatives with respect to the original parameters x_i . In practice, these derivative jets can be computed efficiently with dual numbers.

For a sector map X_s , define the pullbacks

$$U_s(y) = \widehat{\mathcal{U}}(X_s(y)), \quad F_s(y) = \widehat{\mathcal{F}}(X_s(y)). \quad (59)$$

Derivatives with respect to sector variables are obtained by chain rules. For example,

$$\partial_a F_s(y) = \sum_i \frac{\partial \widehat{\mathcal{F}}}{\partial x_i}(X_s(y)) \frac{\partial X_{s,i}}{\partial y_a}(y), \quad (60)$$

and

$$\begin{aligned} \partial_a \partial_b F_s(y) &= \sum_{i,j} \frac{\partial^2 \widehat{\mathcal{F}}}{\partial x_i \partial x_j}(X_s(y)) \frac{\partial X_{s,i}}{\partial y_a}(y) \frac{\partial X_{s,j}}{\partial y_b}(y) \\ &\quad + \sum_i \frac{\partial \widehat{\mathcal{F}}}{\partial x_i}(X_s(y)) \frac{\partial^2 X_{s,i}}{\partial y_a \partial y_b}(y). \end{aligned} \quad (61)$$

Higher derivatives are obtained similarly, or more directly by propagating dual-number jets through the known sector map and then through the black-box polynomial evaluator.

The object to construct is the regular coefficient ϕ_s defined by

$$F_s(y) = M_s(y) \phi_s(y), \quad (62)$$

where M_s is the expected monomial for that sector. Once ϕ_s is available, the sector integrand is

$$I_s^{\text{ind}}(y) = J_s(y) U_s(y)^{3-D} M_s(y)^{-\alpha} \phi_s(y)^{-\alpha}. \quad (63)$$

At interior points this is pointwise identical to direct substitution, since

$$\phi_s(y) = \frac{\widehat{\mathcal{F}}(X_s(y))}{M_s(y)} \quad (64)$$

there. For the endpoint decomposition before any monomial extraction, the black-box form of the exact triangle is

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} \widehat{\mathcal{U}}(X_{S_a}(t, z))^{3-D} \widehat{\mathcal{F}}(X_{S_a}(t, z))^{-\alpha}. \quad (65)$$

When $m = 0$, and hence $\widehat{\mathcal{F}}(X_{S_a}(t, z)) = t^2 z \phi_{S_a}(t, z)$, this same equality becomes

$$C_0(s; 0) = -\Gamma(1 + \epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} \widehat{\mathcal{U}}(X_{S_a}(t, z))^{-1+2\epsilon} \times (t^2 z)^{-1-\epsilon} \phi_{S_a}(t, z)^{-1-\epsilon}. \quad (66)$$

3.1 Using Cauchy remainder formula

Suppose

$$M_s(y) = \prod_{a=1}^n y_a^{r_a}, \quad r_a \in \mathbb{N}. \quad (67)$$

If F_s is divisible by M_s , then repeated use of the Cauchy form of the Taylor remainder gives

$$\phi_s(y) = \prod_{a:r_a>0} \left[\frac{1}{(r_a-1)!} \int_0^1 d\lambda_a (1-\lambda_a)^{r_a-1} \right] \left(\prod_{a:r_a>0} \partial_{y_a}^{r_a} \right) F_s(y^{(\lambda)}), \quad (68)$$

where

$$y_a^{(\lambda)} = \begin{cases} \lambda_a y_a, & r_a > 0, \\ y_a, & r_a = 0. \end{cases} \quad (69)$$

This formula does not require opening $\widehat{\mathcal{F}}$. The derivatives of $F_s = \widehat{\mathcal{F}} \circ X_s$ are supplied by the chain rules above.

For the first endpoint sector,

$$M_{S_1}(t, z) = t^2 z. \quad (70)$$

Introduce dummy variables T, Z and write

$$F_{S_1}(T, Z) = \widehat{\mathcal{F}} \left(1 - T, \frac{T}{1+Z}, \frac{TZ}{1+Z} \right). \quad (71)$$

Then

$$\phi_{S_1}^C(t, z) = \int_0^1 d\lambda (1-\lambda) \int_0^1 d\mu \left[\partial_T^2 \partial_Z F_{S_1}(T, Z) \right]_{T=\lambda t, Z=\mu z}. \quad (72)$$

Equivalently, keeping the black box unopened,

$$\phi_{S_1}^C(t, z) = \int_0^1 d\lambda (1-\lambda) \int_0^1 d\mu \partial_T^2 \partial_Z \widehat{\mathcal{F}} \left(1 - T, \frac{T}{1+Z}, \frac{TZ}{1+Z} \right) \Big|_{T=\lambda t, Z=\mu z}. \quad (73)$$

The second endpoint sector is obtained with

$$F_{S_2}(T, Z) = \widehat{\mathcal{F}} \left(1 - T, \frac{TZ}{1+Z}, \frac{T}{1+Z} \right), \quad (74)$$

and the same Cauchy formula,

$$\phi_{S_2}^C(t, z) = \int_0^1 d\lambda (1-\lambda) \int_0^1 d\mu \left[\partial_T^2 \partial_Z F_{S_2}(T, Z) \right]_{T=\lambda t, Z=\mu z}. \quad (75)$$

Using these Cauchy-remainder coefficients, the complete massless triangle is

$$C_0(s; 0) = -\Gamma(1 + \epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} \widehat{\mathcal{U}}(X_{S_a}(t, z))^{-1+2\epsilon} \times (t^2 z)^{-1-\epsilon} [\phi_{S_a}^C(t, z)]^{-1-\epsilon}. \quad (76)$$

If we momentarily open the explicit massless triangle polynomial, then

$$F_{S_1}(T, Z) = (-s - i0) \frac{T^2 Z}{(1+Z)^2}. \quad (77)$$

Therefore

$$\partial_T^2 \partial_Z F_{S_1}(T, Z) = 2(-s - i0) \frac{1-Z}{(1+Z)^3}. \quad (78)$$

The explicit (λ, μ) -integrand is

$$(1-\lambda) 2(-s - i0) \frac{1-\mu z}{(1+\mu z)^3}. \quad (79)$$

Consequently,

$$\begin{aligned} \phi_{S_1}^C(t, z) &= \int_0^1 d\lambda (1-\lambda) \int_0^1 d\mu 2(-s - i0) \frac{1-\mu z}{(1+\mu z)^3} \\ &= \frac{-s - i0}{(1+z)^2}, \end{aligned} \quad (80)$$

as in the direct approach. The same computation holds in sector S_2 . Hence the explicit Cauchy-remainder reconstruction of the massless triangle is

$$\begin{aligned} C_0(s; 0) &= -\Gamma(1 + \epsilon) \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz \frac{t}{(1+z)^2} (t^2 z)^{-1-\epsilon} \left(\frac{-s - i0}{(1+z)^2} \right)^{-1-\epsilon} \\ &= -2\Gamma(1 + \epsilon) (-s - i0)^{-1-\epsilon} \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} (1+z)^{2\epsilon}. \end{aligned} \quad (81)$$

The auxiliary integrals are well behaved because the assumed monomial divisibility implies that $F_s = M_s \phi_s$ with ϕ_s regular at the sector boundaries. The Cauchy-remainder integrands are derivatives of a regular pullback evaluated on a compact domain. In this explicit sector,

$$\frac{1-\mu z}{(1+\mu z)^3} \quad (82)$$

is bounded for $0 \leq \mu, z \leq 1$.

3.2 Using subtraction terms

The alternative is to avoid the auxiliary Cauchy-remainder integrals. At interior points one simply evaluates

$$\phi_s(y) = \frac{\widehat{\mathcal{F}}(X_s(y))}{M_s(y)}. \quad (83)$$

Boundary values are supplied by the continuous extension, using derivatives. For the first endpoint sector,

$$\phi_{S_1}(0, z) = \frac{1}{2z} [\partial_T^2 F_{S_1}(T, z)]_{T=0}, \quad (84)$$

$$\phi_{S_1}(t, 0) = \frac{1}{t^2} [\partial_Z F_{S_1}(t, Z)]_{Z=0}, \quad (85)$$

$$\phi_{S_1}(0, 0) = \frac{1}{2} [\partial_T^2 \partial_Z F_{S_1}(T, Z)]_{T=0, Z=0}. \quad (86)$$

Again, the derivatives of F_{S_1} are not symbolic derivatives of an opened polynomial. They are derivatives of $\widehat{\mathcal{F}} \circ X_{S_1}$, obtained from the black-box derivative jets and the chain rule.

For $D = 4 - 2\epsilon$, write each factorized endpoint integrand as

$$I_{S_a}^{\text{ind}}(t, z) = t^{-1-2\epsilon} z^{-1-\epsilon} G_{S_a}(t, z), \quad a = 1, 2, \quad (87)$$

where

$$G_{S_a}(t, z) = \frac{1}{(1+z)^2} U_{S_a}(t, z)^{-1+2\epsilon} \phi_{S_a}(t, z)^{-1-\epsilon}. \quad (88)$$

The two-dimensional endpoint subtraction identity is

$$\begin{aligned} & \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} G(t, z) \\ &= \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} [G(t, z) - G(0, z) - G(t, 0) + G(0, 0)] \\ & \quad - \frac{1}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [G(0, z) - G(0, 0)] \\ & \quad - \frac{1}{\epsilon} \int_0^1 dt t^{-1-2\epsilon} [G(t, 0) - G(0, 0)] + \frac{G(0, 0)}{2\epsilon^2}. \end{aligned} \quad (89)$$

Each bracket is less singular at the corresponding endpoint. This is the subtraction-term alternative to constructing ϕ_s with auxiliary integrals. Applied sector by sector, it gives the full massless triangle as

$$\begin{aligned} C_0(s; 0) = -\Gamma(1+\epsilon) \sum_{a=1}^2 \left\{ \int_0^1 dt \int_0^1 dz t^{-1-2\epsilon} z^{-1-\epsilon} \left[G_{S_a}(t, z) - G_{S_a}(0, z) \right. \right. \\ \left. \left. - G_{S_a}(t, 0) + G_{S_a}(0, 0) \right] \right. \\ \left. - \frac{1}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [G_{S_a}(0, z) - G_{S_a}(0, 0)] \right. \\ \left. - \frac{1}{\epsilon} \int_0^1 dt t^{-1-2\epsilon} [G_{S_a}(t, 0) - G_{S_a}(0, 0)] + \frac{G_{S_a}(0, 0)}{2\epsilon^2} \right\}. \end{aligned} \quad (90)$$

For the explicit massless triangle in sector S_1 ,

$$U_{S_1}(t, z) = 1, \quad \phi_{S_1}(t, z) = \frac{-s - i0}{(1+z)^2}. \quad (91)$$

Thus

$$G_{S_1}^{\text{exp}}(t, z) = (-s - i0)^{-1-\epsilon} (1+z)^{2\epsilon}. \quad (92)$$

This function is independent of t . Therefore

$$G_{S_1}^{\text{exp}}(t, z) - G_{S_1}^{\text{exp}}(0, z) - G_{S_1}^{\text{exp}}(t, 0) + G_{S_1}^{\text{exp}}(0, 0) = 0, \quad (93)$$

and

$$G_{S_1}^{\text{exp}}(t, 0) - G_{S_1}^{\text{exp}}(0, 0) = 0. \quad (94)$$

The only non-trivial subtraction piece is

$$G_{S_1}^{\text{exp}}(0, z) - G_{S_1}^{\text{exp}}(0, 0) = (-s - i0)^{-1-\epsilon} [(1+z)^{2\epsilon} - 1]. \quad (95)$$

The explicitly subtracted result for one endpoint sector is therefore

$$\begin{aligned} & \int_0^1 dt \int_0^1 dz I_{S_1}^{\text{dir}}(t, z) \\ &= -\frac{(-s - i0)^{-1-\epsilon}}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [(1+z)^{2\epsilon} - 1] + \frac{(-s - i0)^{-1-\epsilon}}{2\epsilon^2}. \end{aligned} \quad (96)$$

The second sector gives the same contribution, so the full massless triangle is

$$C_0(s; 0) = -2\Gamma(1 + \epsilon) \left\{ -\frac{(-s - i0)^{-1-\epsilon}}{2\epsilon} \int_0^1 dz z^{-1-\epsilon} [(1+z)^{2\epsilon} - 1] + \frac{(-s - i0)^{-1-\epsilon}}{2\epsilon^2} \right\}. \quad (97)$$

This expression was obtained without requiring any symbolic expansion in an implementation: only X_{S_a} , J_{S_a} , the known monomial $t^2 z$, and black-box values and derivative limits of $\hat{\mathcal{U}}$ and $\hat{\mathcal{F}}$ are required in each endpoint sector.

4 Python prototype in FSD

The folder `FSD` contains a first implementation of the construction above. The executable entry point is

$$\text{FSD/FSD.py}. \quad (98)$$

It uses a local Python virtual environment,

$$\text{FSD/.venv}, \quad (99)$$

and a local checkout of `OneLOopBridge`,

$$\text{FSD/external/OneLOopBridge}. \quad (100)$$

The benchmark value is obtained from the Python binding

$$\text{oneloop_bridge.three_point}(0, 0, s, m2, m2, m2). \quad (101)$$

4.1 Separation of prepared sectors and processing

The code mirrors the split used in this note. First, endpoint sectors are prepared independently of the Symanzik polynomials. Each sector stores only the data

$$\{X_s, J_s, M_s\}. \quad (102)$$

For the current prototype these are just S_1 and S_2 . Threshold sectors are intentionally not implemented yet.

The processing layer then receives the prepared sectors together with two black-box Symbolica evaluators,

$$\hat{\mathcal{U}}(x_0, x_1, x_2; s, m^2), \quad \hat{\mathcal{F}}(x_0, x_1, x_2; s, m^2). \quad (103)$$

After these evaluators are built, the processing code does not inspect or symbolically rewrite the expressions for $\mathcal{U}$ or $\mathcal{F}$. In particular, ϕ_s is obtained from

$$\phi_s(t, z) = \frac{\hat{\mathcal{F}}(X_s(t, z))}{M_s(t, z)} \quad (104)$$

at interior points, and from Symbolica dual-number derivatives at boundary points. The dual-number jets are propagated through the known map X_s and then through the black-box evaluator for $\hat{\mathcal{F}}$. This realizes the chain-rule construction described in the previous sections.

4.2 Havana sampling

The prototype uses Symbolica’s Havana integrator as a sampler rather than as a single black-box scalar integrator. One adaptive grid is used for all endpoint-sector contributions:

$$\text{discrete sector id} \quad \otimes \quad (t, z) \in [0, 1]^2. \quad (105)$$

Lower-support subtraction terms are localized into this same two-dimensional domain by multiplying by a unit-density dummy integration over the unused variable. Thus the z -edge term is represented as a function of (t, z) which is independent of t , and the corner term is represented as a constant function of (t, z) . This avoids maintaining separate Havana grids for two-dimensional, one-dimensional, and zero-dimensional pieces.

For every batch, the code samples points from the single grid, evaluates all Laurent coefficients on those same points, and fills its own accumulators. The Havana grid is trained only with the absolute value of the finite-part integrand. In Symbolica’s sampled representation, the first entry of `sample.weights` is the full Monte Carlo weight used in the manual accumulator.

The integration loop reports progress with `progressbar2`. The bar shows the current iteration, the accumulated number of sampled points against the nominal target $N_{\text{iter}}N_{\text{batch}}$, the live relative Monte Carlo error maxed over the Laurent coefficients, and, when a benchmark is available, the live pull maxed over the Laurent coefficients. The live pull is computed after applying the same Gamma and prefactor conventions used in the final output table.

4.3 Implemented modes and conventions

The command-line interface supports two modes. In massive mode, with $m > 0$ and $s < 4m^2$, the integral is finite and the endpoint monomial factorization is not applied. The implementation evaluates

$$C_0(s; m^2) = - \sum_{a=1}^2 \int_0^1 dt \int_0^1 dz J_{S_a}(t, z) \widehat{\mathcal{U}}(X_{S_a}(t, z))^{-1} \widehat{\mathcal{F}}(X_{S_a}(t, z))^{-1}. \quad (106)$$

In massless mode, the implementation uses the subtraction formula of Section 3.2 and returns the coefficients of

$$C_0(s; 0) = \frac{C_{-2}}{\epsilon^2} + \frac{C_{-1}}{\epsilon} + C_0 + \mathcal{O}(\epsilon). \quad (107)$$

The default output convention is the OneLOopBridge convention. In this convention the universal Gamma/Euler factors are stripped from the displayed Laurent coefficients. For the massless endpoint representation this also amounts to removing the finite convention term

$$\frac{\pi^2}{6} C_{-2} \quad (108)$$

from the direct sector-parametric finite coefficient. With this convention, the prototype compares directly to the values returned by OneLOopBridge. The separate command-line option `--prefactor-convention` controls only the overall scalar-integral normalization shown in the output table. The default value `raw` displays the raw OneLOopBridge normalization. The value `feynman` multiplies both the FastSecDec coefficients and the benchmark coefficients by

$$\text{TO_FEYNMAN} = -\frac{1}{16\pi^2}, \quad (109)$$

which is the OneLOopBridge factor converting raw scalar-integral coefficients to its Feynman-normalized convention. In either display convention, values with Monte Carlo uncertainty are printed with the two-significant-digit parenthesis notation.

4.4 Implemented box example

The prototype also contains a finite massive scalar box example with massless external legs,

$$D_0(0, 0, 0, 0, s_{12}, s_{23}; m^2). \quad (110)$$

The benchmark call is

$$\text{oneloop_bridge.four_point}(0, 0, 0, 0, s_{12}, s_{23}, m^2, m^2, m^2, m^2). \quad (111)$$

For equal internal masses, the simplex polynomial used by the implementation is

$$\mathcal{U} = x_0 + x_1 + x_2 + x_3, \quad \mathcal{F} = m^2 \mathcal{U}^2 - s_{12} x_0 x_2 - s_{23} x_1 x_3 - i0. \quad (112)$$

After imposing $\mathcal{U} = 1$, the finite four-point integral in $D = 4$ is

$$D_0 = \sum_{b=0}^3 \int_0^1 dy_0 \int_0^1 dy_1 \int_0^1 dy_2 J_b(y) \hat{\mathcal{F}}(X_b(y))^{-2}. \quad (113)$$

Here b labels the primary sector in which x_b is the largest Feynman parameter. In sector b ,

$$x_b = \frac{1}{1 + y_0 + y_1 + y_2}, \quad x_{j \neq b} = \frac{y_j}{1 + y_0 + y_1 + y_2}, \quad (114)$$

with the three y_j 's assigned in increasing order to the non-dominant parameters. The Jacobian is

$$J_b(y) = \frac{1}{(1 + y_0 + y_1 + y_2)^4}. \quad (115)$$

The same code path now also implements the massless endpoint-subtracted box. For $m = 0$, each primary sector has the form

$$\mathcal{F}(X_b(y)) = \frac{A_b y_\ell + B_b y_r y_s}{(1 + y_0 + y_1 + y_2)^2}, \quad (116)$$

where one of the two bilinears $x_0 x_2$ and $x_1 x_3$ contains the dominant parameter and is therefore linear in the primary variables. The implementation does not need the symbolic values of A_b and B_b ; it only uses the known positions of the linear variable y_ℓ and product variables y_r, y_s . The three secondary sectors are

$$L: \quad y_r = uv, \quad y_\ell = u, \quad y_s = w, \quad M_L = u, \quad (117)$$

$$P: \quad y_r = u, \quad y_s = v, \quad y_\ell = uvw, \quad M_P = uv, \quad (118)$$

$$Q: \quad y_r = u, \quad y_\ell = uv, \quad y_s = vw, \quad M_Q = uv. \quad (119)$$

The secondary Jacobian equals the displayed monomial in each case, so the regular prefactor after monomial extraction is again

$$\frac{1}{(1 + y_0(u, v, w) + y_1(u, v, w) + y_2(u, v, w))^4}. \quad (120)$$

The residual coefficient

$$\phi_{b\alpha}(u, v, w) = \frac{\mathcal{F}(X_{b\alpha}(u, v, w))}{M_\alpha(u, v, w)} \quad (121)$$

is evaluated by the black-box $\mathcal{F}$ away from the endpoint and by dual-number Taylor coefficients at $u = 0$ and $v = 0$. The one-monomial sector L uses the one-variable subtraction formula, while the P and Q sectors use the two-variable subtraction formula.

The raw coefficients produced directly by these massless box sectors are converted to the OneLOopBridge stripped convention as

$$D_{-2} = A_{-2}, \quad D_{-1} = A_{-1} + A_{-2}, \quad D_0 = A_0 + A_{-1} + \frac{\pi^2}{6} A_{-2}. \quad (122)$$

The current massless box command validates Euclidean kinematics, $s_{12} < 0$ and $s_{23} < 0$. Time-like or threshold kinematics would require a contour deformation or threshold regularization, which is outside the present prototype.

4.5 Example commands

The massive below-threshold check can be run with

```
FSD/.venv/bin/python FSD/FSD.py --s 1.0 --m 1.0 --mode massive. (123)
```

The same check in the Feynman-normalized prefactor convention can be displayed with

```
FSD/.venv/bin/python FSD/FSD.py --s 1.0 --m 1.0 --mode massive
--prefactor-convention feynman. (124)
```

The massless Euclidean check can be run with

```
FSD/.venv/bin/python FSD/FSD.py --s -1.0 --m 0.0 --mode massless. (125)
```

The timelike command with $s > 0$ is intentionally rejected in the current prototype because no contour deformation or threshold regularization is implemented. The massive box example can be run with

```
FSD/.venv/bin/python FSD/FSD.py --integral box --s12 0.5 --s23 0.7 --m 1.0. (126)
```

The massless box check can be run with

```
FSD/.venv/bin/python FSD/FSD.py --integral box --s12 -1.0 --s23 -2.0 --m 0.0. (127)
```

The corresponding timelike massless box command is rejected for the same reason as the timelike triangle. The implementation intentionally imports neither `scipy` nor `sympy`; all symbolic handling is done through Symbolica evaluators.

5 Implementation of the modular FSD_v2 prototype

The sibling folder `FSD_v2` is the modular reimplementaion of the prototype. Its implementation is organized so that the black-box rule is visible directly in the code layout:

- `FSD.py` is only the command-line steering layer. It parses options, validates kinematics, builds the request object, prints summaries, calls the benchmark, launches the integration, and renders the final table.
- `sectors_generator.py` owns the declarative sector data: sector maps X_s , regular Jacobians J_s , endpoint monomials M_s , singular axes, and generated Symbolica callbacks for these expressions.
- `integrand.py` owns the topology definition and the generic `SectorProcessor`. The topology retains $\mathcal{U}$ and $\mathcal{F}$ for display and evaluator construction, but the processor never substitutes a sector map into them symbolically.
- `integrator.py` owns the Havana sampling loop, manual Laurent coefficient accumulation, progress reporting, target-accuracy stopping, and hot-path timing profile.
- `benchmark.py`, `formatting.py`, `definitions.py`, and `utils.py` isolate `OneLoopBridge` access, colored output, shared data containers, and parenthesis uncertainty formatting.

At runtime the processor receives only a `SectorDefinition`, a `TopologyDefinition`, and numeric sample points. For a sector point y it evaluates

$$x = X_s(y), \quad \hat{U}(x), \quad \hat{F}(x), \quad J_s(y), \quad (128)$$

through prebuilt Symbolica evaluators. If the sector has no endpoint monomial, the contribution is evaluated directly. If a monomial is declared, the residual

$$\phi_s(y) = \frac{\hat{\mathcal{F}}(X_s(y))}{M_s(y)} \quad (129)$$

is evaluated by this quotient in the interior. On endpoint samples the same quantity is recovered from the Taylor coefficients of a dualized black-box $\hat{\mathcal{F}}$ evaluator. The dual jets of the known sector map provide the chain rule input, so no symbolic expansion of $\mathcal{F}(X_s(y))$ is performed.

The generic processor then constructs

$$g_s(y; \epsilon) = J_s(y) \hat{\mathcal{U}}(X_s(y))^{p_U+2\epsilon} \phi_s(y)^{-p_F-\epsilon}, \quad (130)$$

and applies the declared localized subtraction formula. Zero, one, and two singular axes are supported explicitly. Stronger endpoint structures raise an error instead of being guessed.

The Havana driver samples one discrete sector dimension and the common continuous sector dimension. Lower-support subtraction pieces are localized into the same highest-dimensional sample space by dummy unit-density dependence on unused variables. Each sampled point therefore contributes to all Laurent coefficients through the same manual accumulator,

$$A_k = \frac{1}{N} \sum_{n=1}^N w_n I_{k,s_n}(y_n), \quad k \in \{-2, -1, 0\}. \quad (131)$$

Havana is trained only on the finite-part magnitude used as an importance function. The adaptive grid is cloned at the beginning of an iteration; batch training samples are accumulated into the clone and merged back into the live grid only at the end of a completed iteration.

The target-accuracy stop is evaluated after each accumulated batch, not only after a full Havana iteration. Thus the final returned coefficients contain all completed batches up to the stopping point. In parallel mode the driver keeps only a worker-sized window of batches in flight, so early stopping does not submit the entire iteration unnecessarily. The option `--batch-size` therefore controls both vectorization size and mid-iteration stopping granularity.

The timing profile reported by the progress bar and final table is a work-time profile:

$$T_{\text{prof}} = T_{\text{Python}} + T_{\text{eval}} + T_{\text{Havana}}. \quad (132)$$

Here T_{eval} is the sum of Symbolica evaluator wall times over all workers, T_{Python} is NumPy and Python glue on workers plus master-side submission, result retrieval, accumulation, and progress rendering, and T_{Havana} is Havana sampling, training accumulation, grid merge, and grid update time. The displayed evaluator average is

$$\langle t_{\text{eval}} \rangle = \frac{10^6 T_{\text{eval}}}{N_{\text{accepted}}} \quad \text{microseconds per sample per worker}, \quad (133)$$

because T_{eval} is already summed over workers.

The current supported commands are, for example,

$$\text{FSD_v2/.venv/bin/python FSD.py --integral triangle --s 1.0 --m 1.0}, \quad (134)$$

$$\text{FSD_v2/.venv/bin/python FSD.py --integral triangle --s -1.0 --m 0.0}, \quad (135)$$

$$\text{FSD_v2/.venv/bin/python FSD.py --integral box --s12 0.5 --s23 0.7 --m 1.0}, \quad (136)$$

$$\text{FSD_v2/.venv/bin/python FSD.py --integral box --s12 -1.0 --s23 -2.0 --m 0.0}. \quad (137)$$

Massless timelike commands, such as `--integral triangle --s 1.0 --m 0.0`, are rejected by validation. OneLoopBridge is required in all runs because the implementation always compares the Laurent coefficients to a benchmark value.